

Ambiente de Datos

Aula 8

SQL – DML – INTERMEDIARIO

Classificação da Linguagem de Banco de Dados

- **Linguagem de Definição de Dados (DDL)** - Linguagem utilizada para definir a estrutura de armazenamento dos dados, também chamada de dicionário de dados ou metadados.
- **Linguagem de Manipulação de Dados (DML)** - Linguagem utilizada para leitura e armazenamento de dados.
- **Linguagem de Controle de Dados (DCL)** - Linguagem utilizada para controle de sessão, permissão, além do controle do próprio sistema de banco de dados.
- **Linguagem de Controle de Transação (TCL)** – Linguagem de controle de Transações (commit, rollback).

Comandos DML

- **INSERT**
- **SELECT**
- **UPDATE**
- **DELETE**



SELECT

Funções Agregadas no SQL Padrão

- Contagem (COUNT)
- Soma (SUM)
- Média (AVG)
- Valor Máximo (MAX)
- Valor Mínimo (MIN)

Função Agregada de Contagem no SQL Padrão

```
SELECT COUNT(*)  
FROM LIVRO
```

COUNT(*)

5

```
SELECT COUNT(CODIGO)  
FROM LIVRO
```

COUNT(CODIGO)

5

```
SELECT COUNT(LANCAMENTO)  
FROM LIVRO
```

COUNT(LANCAMENTO)

4

Função Agregada de Soma no SQL Padrão

```
SELECT SUM(PRECO)  
FROM LIVRO
```

SUM(PRECO)

262.7

Função Agregada de Média Aritmética no SQL Padrão

```
SELECT AVG(PRECO)  
FROM LIVRO
```

AVG(PRECO)

52.54

Função Agregada de Valor Máximo no SQL Padrão

```
SELECT MAX(PRECO)
FROM LIVRO
```

MAX(PRECO)

111.5

Função Agregada de Valor Mínimo no SQL Padrão

```
SELECT MIN(PRECO)
FROM LIVRO
```

MIN(PRECO)

31.2

Consulta com a Cláusula *GROUP BY* no *SQL Padrão*

```
SELECT    COL1,  
            COL2,  
            ...,  
            COLN,  
            FUNCAO1,  
            FUNCAO2,  
            ...,  
            FUNCAON  
  
FROM      NOME_TABELA  
  
WHERE     EXPRESSAO_LOGICA  
  
GROUP BY  COL1, COL2, ..., COLN
```

Consulta com a Cláusula *GROUP BY*

```
SELECT  COD_ASSUNTO, AVG(PRECO)
FROM    LIVRO
GROUP BY COD_ASSUNTO
```

COD_ASSUNTO	AVG(PRECO)
-----	-----
B	39.6
P	70.75
R	42

Consulta com a Cláusula *GROUP BY*

```
SELECT  COD_ASSUNTO, MAX(PRECO)
FROM    LIVRO
WHERE   LANCAMENTO IS NOT NULL
GROUP BY COD_ASSUNTO
```

COD_ASSUNTO	MAX(PRECO)
-------------	------------

B	31.2
---	------

P	111.5
---	-------

R	42
---	----

Consulta com a Cláusula *GROUP BY*

```
SELECT  COD_EDITORA, COD_ASSUNTO, MAX(PRECO)
FROM    LIVRO
GROUP BY COD_EDITORA, COD_ASSUNTO
```

COD_EDITORA	COD_ASSUNTO	MAX(PRECO)
-----	-----	-----
1	B	31.2
1	P	30
2	B	111.5
2	P	48
2	R	42

Consulta com a Cláusula *HAVING* no *SQL Padrão*

```
SELECT    COL1,  
            COL2,  
            ...,  
            COLN,  
            FUNCAO1,  
            FUNCAO2,  
            ...,  
            FUNCAON  
  
FROM      NOME_TABELA  
  
WHERE     EXPRESSAO_LOGICA_WHERE  
  
GROUP BY  COL1, COL2, ..., COLN  
  
HAVING    EXPRESSAO_LOGICA_HAVING
```

Consulta com a Cláusula *HAVING*

```
SELECT    COD_ASSUNTO, AVG (PRECO)
FROM      LIVRO
GROUP BY  COD_ASSUNTO
HAVING    AVG(PRECO) > 50
```

COD_ASSUNTO	AVG(PRECO)
-------------	------------

P	60
---	----

Consulta com a Cláusula *HAVING*

```
SELECT    COD_ASSUNTO, COUNT(*)  
FROM      LIVRO  
GROUP BY  COD_ASSUNTO  
HAVING    COUNT(*) > 1
```

COD_ASSUNTO	COUNT(*)
-------------	----------

B	2
P	2

Consulta com a Cláusula *HAVING*

```
SELECT    COD_ASSUNTO, COUNT(*)  
FROM      LIVRO  
WHERE     DATA_LANCAMENTO IS NOT NULL  
GROUP BY  COD_ASSUNTO  
HAVING    COUNT(*) > 1
```

COD_ASSUNTO	COUNT(*)
-----	-----
P	2

Sintaxe Genérica para Consulta de Dados em *SQL Padrão*

```
SELECT    COL1,  
           COL2,  
           ...,  
           COLN  
FROM      NOME_TABELA1,  
           NOME_TABELA2,  
           ...,  
           NOME_TABELAN  
WHERE     EXPRESSAO_LOGICA_WHERE  
GROUP BY COL1,  
           COL2,  
           ...,  
           COLN  
HAVING    EXPRESSAO_LOGICA_HAVING  
ORDER BY COL1,  
           COL2,  
           ...,  
           COLN
```

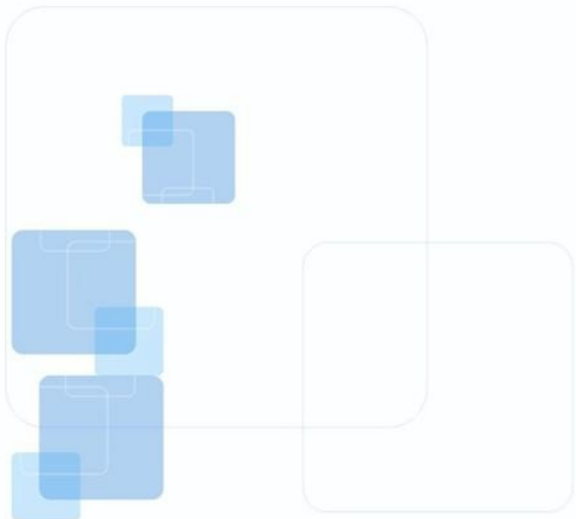
```
SELECT    MARCA_VEICULO,  
           COUNT(*)  
FROM      TB_VEICULO  
WHERE      PRECO > 10000.00  
GROUP BY  MARCA_VEICULO  
HAVING    COUNT(*) > 5  
ORDER BY  MARCA_VEICULO ASC
```

SQL – DML - INTERMEDIARIO

MANIPULAÇÃO DE DADOS COM MAIS DE UMA TABELA
(IMPLEMENTAÇÃO DE OPERADORES RELACIONAIS)

TIPOS DE JUNÇÃO ENTRE TABELAS:

- INNER
- OUTER
- CROSS
- SELF

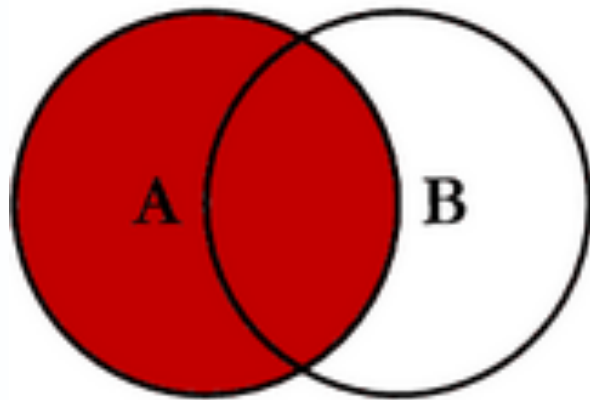


TIPOS DE JUNÇÃO ENTRE TABELAS:

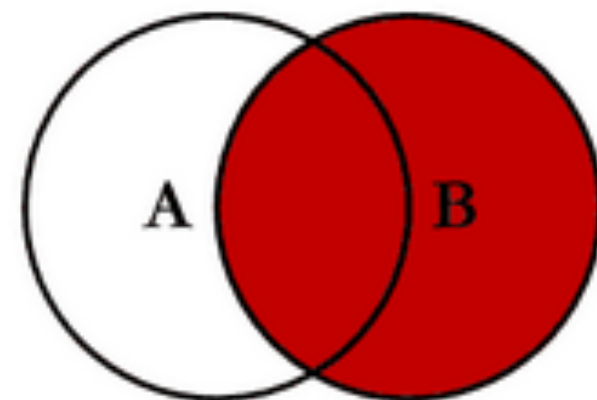
- Inner join
- Outer
 - Left outer join
 - Right outer join
 - Outer Full join
- Cross join
- Self join



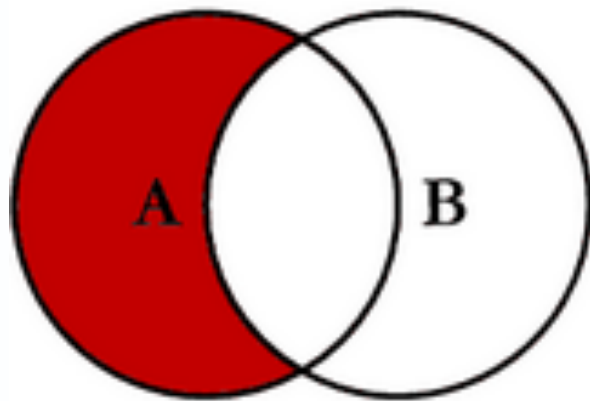
SQL JOINS



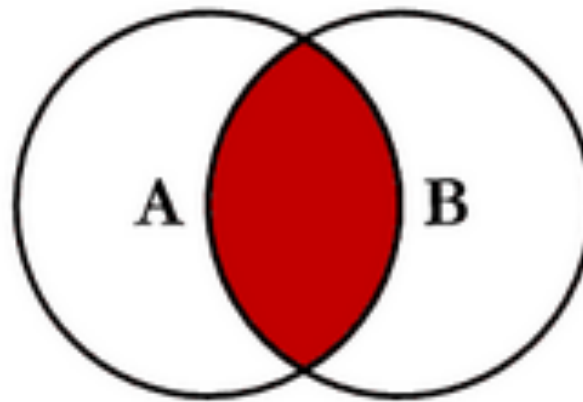
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



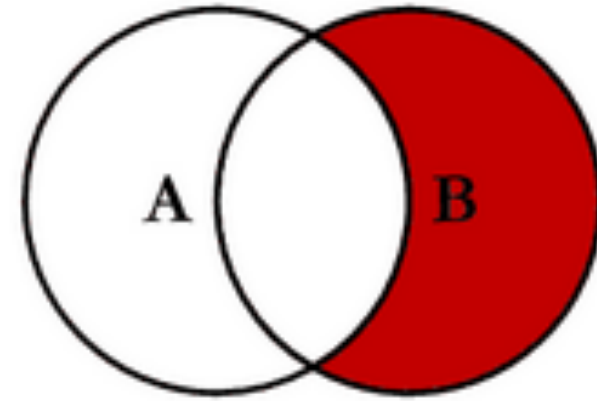
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



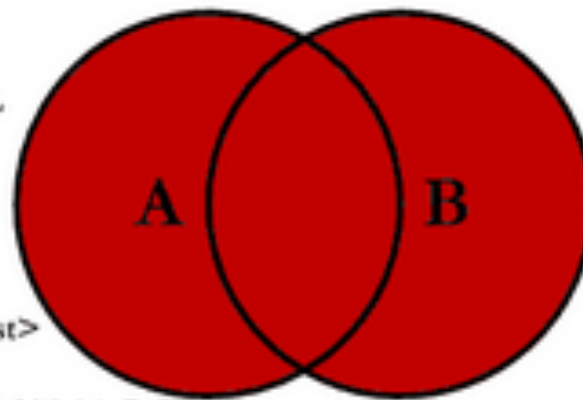
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



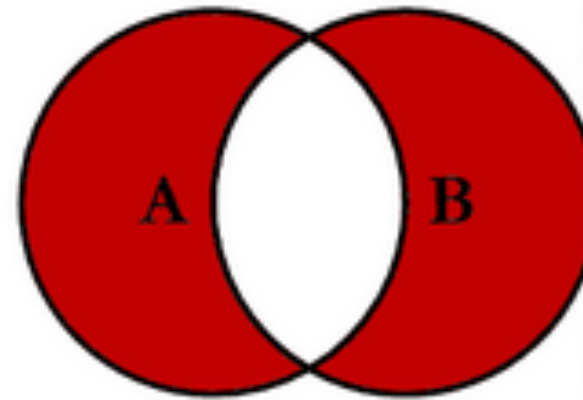
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

SINTAXES EQUIVALENTE:

TABELA_A LEFT JOIN TABELA_B

TABELA_A LEFT **OUTER** JOIN TABELA_B

TABELA_A RIGHT JOIN TABELA_B

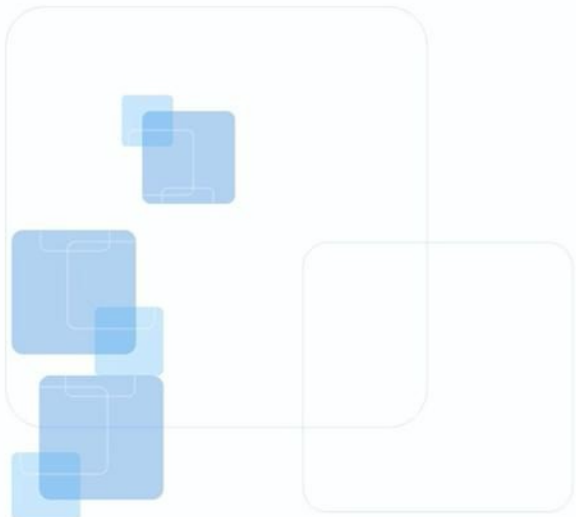
TABELA_A RIGHT **OUTER** JOIN TABELA_B

TABELA_A FULL JOIN TABELA_B

TABELA_A FULL **OUTER** JOIN TABELA_B

TABELA_A **INNER** JOIN TABELA_B

TABELA_A JOIN TABELA_B



TIPOS DE JUNÇÃO ENTRE TABELAS:

FUNCIONARIO

Matricula	INTEGER	NOT NULL
NomeFuncionario	VARCHAR(80)	NOT NULL
CodCargo (FK)	CHAR(2)	NULL

CARGO

CodCargo	CHAR(2)	NOT NULL
NomeCargo	VARCHAR(50)	NOT NULL
ValorCargo	MONEY(10,0)	NOT NULL



	CodCargo	NomeCargo	ValorCargo
1	C1	CAIXA	800,00
2	C2	VENDEDOR	1200,00
3	C3	GERENTE	2400,00

CARGO AS C

	Matricula	NomeFuncionario	CodCargo
1	100	JOÃO	C1
2	110	MARIA	C2
3	120	CARLOS	C1
4	130	TADEU	NULL

FUNCIONARIO AS F

INNER JOIN

- Objetivo: juntar duas ou mais tabelas por coincidência.
- Exemplo: Para cada linha da tabela FUNCINARIO queremos o CARGO correspondente que internamente (INNER), em seus valores de atributos, coincidam.

```
SELECT F.NomeFuncionario  
       ,C.NomeCargo  
FROM   CARGO           AS C  
       INNER JOIN      FUNCIONARIO AS F ON (F.CodCargo = C.CodCargo)
```

Results Messages

	NomeFuncionario	NomeCargo
1	JOÃO	CAIXA
2	MARIA	VENDEDOR
3	CARLOS	CAIXA

LEFT OUTER JOIN

- Observando a tabela FUNCIONARIO notamos que o funcionário *Tadeu* não possui cargo associado a ele.
- Se desejarmos listar **todos os funcionários com seus respectivos cargos, incluindo os funcionários sem cargos**, a exemplo de Tadeu, poderíamos usar todo o poder da junção INNER JOIN adicionando ainda OUTER(EXTERNOS/OUTROS) Funcionários que não fazem parte do INNER JOIN, justamente àqueles sem cargos, como Tadeu.
- Podemos conseguir esse feito com a junção FUNCIONARIO/CARGO através da declaração **FUNCIONARIO OUTER LEFT JOIN CARGO**, que promove a junção interna (INNER) de todos os funcionários a cargos e **lista ainda** outros (EXTERNOS/OUTER) não associados.

LEFT OUTER JOIN

```
SELECT F.nomeFuncionario  
      ,C.nomeCargo  
FROM      FUNCIONARIO AS F  
      LEFT OUTER JOIN CARGO      AS C ON ( F.codCargo = C.codCargo )
```

Results Messages

	nomeFuncionario	nomeCargo
1	JOÃO	CAIXA
2	MARIA	VENDEDOR
3	CARLOS	CAIXA
4	TADEU	NULL

RIGHT OUTER JOIN

- Observando a tabela CARGO notamos que o cargo GERENTE, com código C3, não é referenciado/associado por/a nenhum funcionário na tabela FUNCIONARIO.
- Se desejarmos listar todos os CARGOS e seus respectivos FUNCIONARIOS, incluindo os CARGOS sem FUNCIONÁRIOS, poderíamos usar a junção RIGHT OUTER JOIN.

```
SELECT F.nomeFuncionario  
      ,C.nomeCargo  
  
FROM      FUNCIONARIO AS F  
      RIGHT OUTER JOIN CARGO      AS C  ON ( F.codCargo = C.codCargo )
```

Results Messages

	nomeFuncionario	nomeCargo
1	JOÃO	CAIXA
2	CARLOS	CAIXA
3	MARIA	VENDEDOR
4	NULL	GERENTE

FULL OUTER JOIN

- Aqui juntamos o poder das junções (JOIN) internas (INNER), a listagem de todas as outras linhas não associadas, tanto do lado direito (RIGHT) da junção como do lado ESQUERDO (LEFT).

```
SELECT F.nomeFuncionario  
       ,C.nomeCargo  
FROM   FUNCIONARIO AS F  
       FULL OUTER JOIN CARGO AS C ON ( F.codCargo = C.codCargo )
```

Results

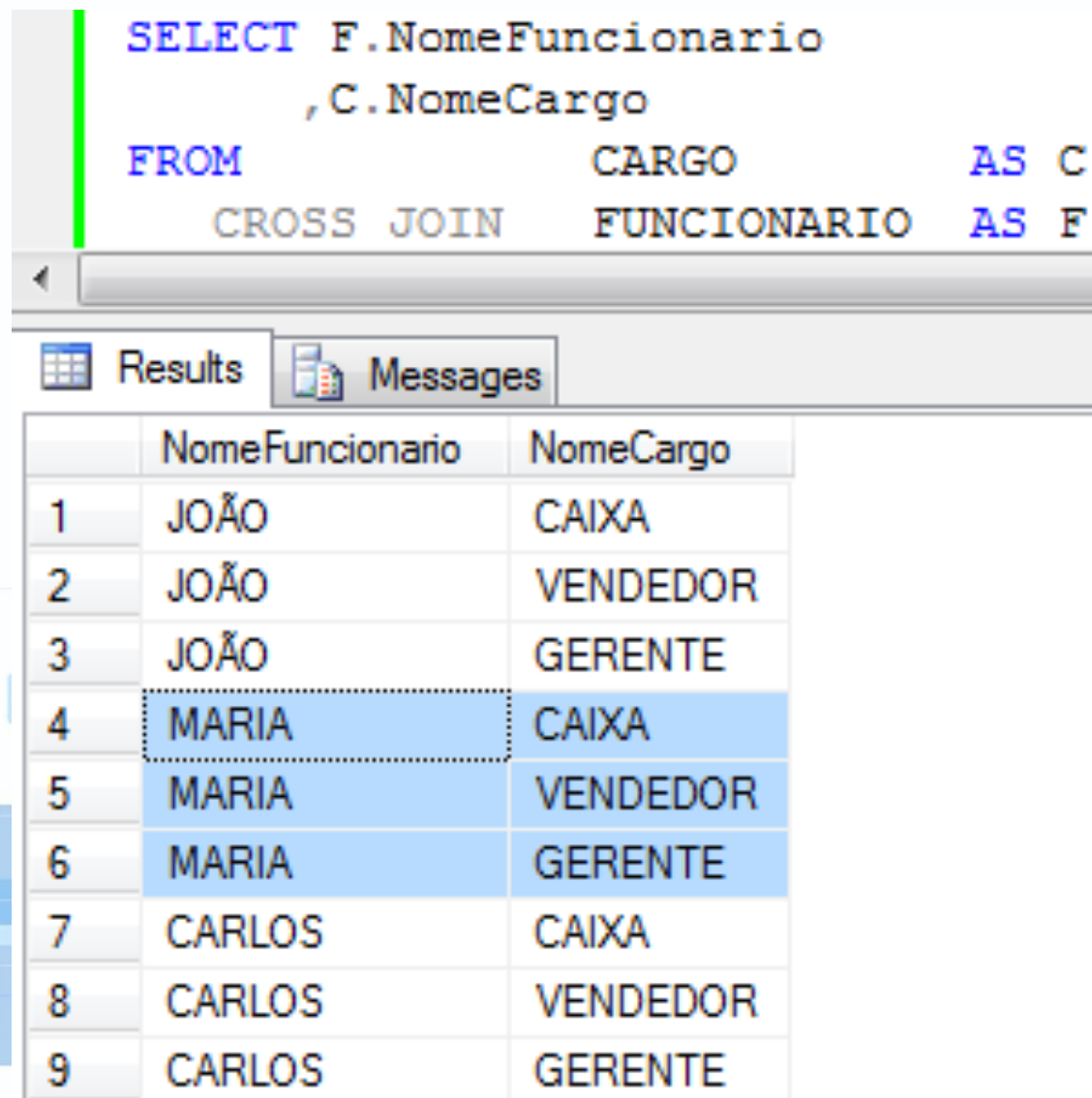
Messages

	nomeFuncionario	nomeCargo
1	JOÃO	CAIXA
2	MARIA	VENDEDOR
3	CARLOS	CAIXA
4	TADEU	NULL
5	NULL	GERENTE

CROSS JOIN

- Quando queremos juntar duas ou mais tabelas por cruzamento. Ou seja, para cada linha da tabela FUNCIONARIO queremos todos os CARGOS ou vice-versa.

```
SELECT F.NomeFuncionario
       ,C.NomeCargo
FROM   CARGO      AS C
CROSS JOIN FUNCIONARIO AS F
```



	NomeFuncionario	NomeCargo
1	JOÃO	CAIXA
2	JOÃO	VENDEDOR
3	JOÃO	GERENTE
4	MARIA	CAIXA
5	MARIA	VENDEDOR
6	MARIA	GERENTE
7	CARLOS	CAIXA
8	CARLOS	VENDEDOR
9	CARLOS	GERENTE

SELF JOIN

- A relação de dados da mesma tabela, estabelecida através de um auto-relacionamento é conhecido como self join.
- Exemplo:
 - ```
SELECT FUN.NM_FUNCIONARIO, SUP.NM_FUNCIONARIO AS NM_SUPERVISOR
FROM FUNCIONARIO FUN,
 FUNCIONARIO SUP
WHERE
 FUN.NR_FUNCIONARIO = SUP.NR_SUPERVISOR AND FUN.NR_SUPERVISOR IS NOT NULL
```

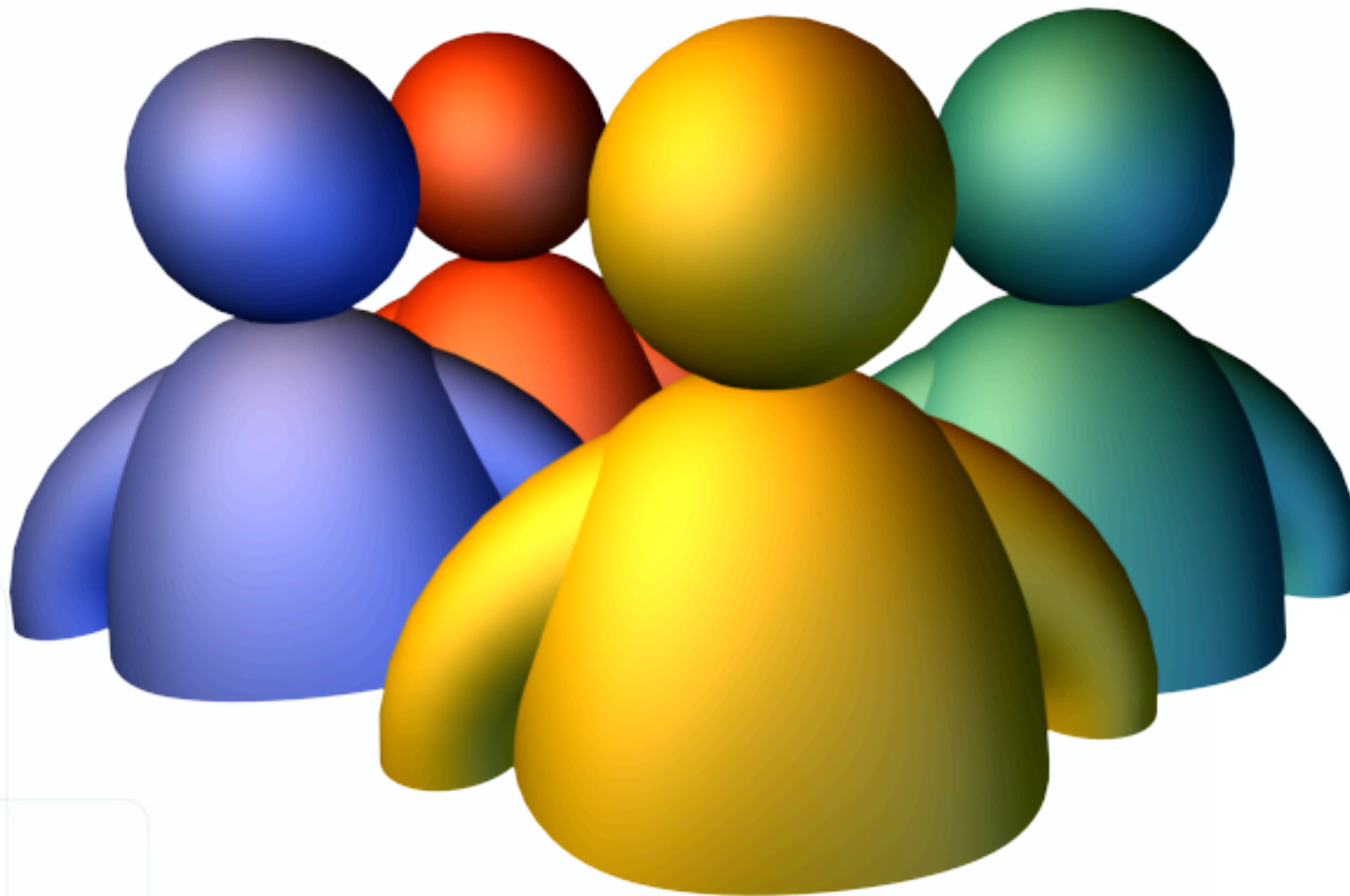


# CENÁRIO PRÁTICO

# EXERCÍCIOS DE FIXAÇÃO (SQL) – UTILIZE O BANCO DE DADOS SAKILA DO MYSQL

1. Liste os filmes que possuem taxa de locação (rental\_rate) superior a R\$ 3,00;
2. Liste os atores que atuaram nos filmes disponíveis.
3. Apresente a quantidade de filmes por Duração da Locação (Rental\_Duration) (Utilize o GROUP BY).
4. Liste o valor do filme com maior preço de locação (rental\_rate)
5. Apresente os filmes que possuem no título a palavra 'WEST';
6. Liste os clientes que não estão ativos;

# Dúvidas?





Obrigado e até a Próxima Aula!

